K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Assignment Title:

Multiclass Image Classification Using Convolutional Neural Networks and Transfer Learning

Course:

Deep Learning Architectures and Techniques (ETMMDL274)

Instructor:

Dr. Shahid Ahmad Wani

Name:

Jitin Kumar Sengar

Student ID:

2501940073

Submission Date:

February 5, 2026

1. Introduction

Image classification is one of the most important applications of deep learning. It involves assigning a label or category to an image based on its visual content. Many real-world systems such as autonomous vehicles, medical diagnosis systems, and facial recognition technologies rely heavily on accurate image classification.

Convolutional Neural Networks (CNNs) are specifically designed for image data. Unlike traditional machine learning models, CNNs automatically learn spatial patterns such as edges, shapes, and textures from images. This makes them highly effective for computer vision tasks.

In this laboratory assignment, a **multiclass image classification model** was implemented using the **CIFAR-10 dataset**. Two different approaches were used:

1. **Custom Convolutional Neural Network (CNN)**
2. **Transfer Learning using a pretrained MobileNetV2 model**

The objective of this experiment was to understand how CNN architectures work and to compare the performance of a custom CNN model with a pretrained model using transfer learning.

2. Dataset Description

The dataset used in this experiment is the **CIFAR-10 dataset**, which is a widely used benchmark dataset for image classification tasks.

The dataset contains **60,000 colored images** belonging to **10 different classes**. Each image has a size of **32 × 32 pixels with three color channels (RGB)**.

The dataset is divided into:

Dataset	Number of Images
Training Dataset	50,000
Test Dataset	10,000

The classes present in the CIFAR-10 dataset are:

Class Index	Class Name
0	Airplane
1	Automobile
2	Bird
3	Cat
4	Deer
5	Dog
6	Frog
7	Horse
8	Ship
9	Truck

Each class contains different object images, making the dataset suitable for **multiclass image classification tasks**.

3. Image Preprocessing

Before training the neural network models, several preprocessing steps were applied to prepare the dataset.

3.1 Normalization

Pixel values in images range from **0 to 255**. Neural networks perform better when inputs are scaled to a smaller range. Therefore, all pixel values were normalized by dividing them by **255**, converting the values to the range **0–1**.

Normalization helps improve training stability and speeds up the learning process.

3.2 Train–Validation Split

The training dataset was divided into:

Dataset	Images
Training Set	40,000
Validation Set	10,000

The validation dataset was used during training to monitor model performance and detect overfitting.

3.3 Data Augmentation

Data augmentation was applied to increase the diversity of training images and reduce overfitting.

The following augmentation techniques were used:

- Random rotation
- Horizontal flipping
- Width shifting
- Height shifting
- Zooming

These transformations allow the neural network to learn robust image features from different variations of the same image.

4. Custom CNN Architecture

A custom Convolutional Neural Network was designed to classify the CIFAR-10 images.

The architecture of the custom CNN consists of the following layers:

Layer	Description
Conv2D (32 filters)	Extracts low-level image features
MaxPooling	Reduces spatial dimensions
Conv2D (64 filters)	Extracts more complex patterns
MaxPooling	Reduces feature map size
Conv2D (128 filters)	Captures deeper visual features
MaxPooling	Further reduces spatial dimensions
Flatten	Converts feature maps to vector
Dense Layer (128 neurons)	Fully connected classification layer
Output Layer (Softmax)	Predicts probability for 10 classes

Activation Function

The **ReLU activation function** was used in the hidden layers because it improves training efficiency and prevents vanishing gradient problems.

The **Softmax activation function** was used in the output layer to generate probability scores for the 10 classes.

Model Training

The custom CNN model was trained using:

Parameter	Value
Loss Function	Sparse Categorical Cross-Entropy
Optimizer	Adam
Epochs	15
Batch Size	64

5. Training Results

In addition to the custom CNN, **transfer learning** was applied using the **MobileNetV2 pretrained model**.

Transfer learning allows us to use a neural network that has already been trained on a large dataset such as **ImageNet**, which contains millions of images.

Instead of training the entire network from scratch, the pretrained layers of MobileNetV2 were used as a **feature extractor**.

Transfer Learning Steps

1. Load the pretrained **MobileNetV2 model**
2. Remove the top classification layer
3. Freeze the pretrained layers
4. Add new fully connected layers for CIFAR-10 classification
5. Train the new classification layers on the dataset

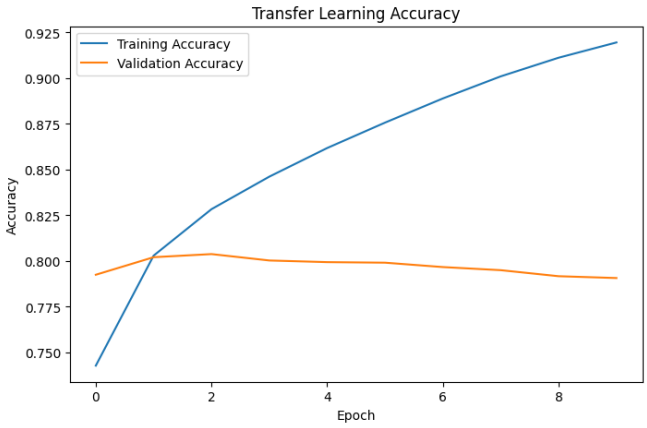
Since MobileNetV2 requires larger images, the CIFAR-10 images were resized to **96 × 96 pixels**.

This approach allows the model to leverage powerful visual features learned from large datasets.

6. Training Results and Visualization

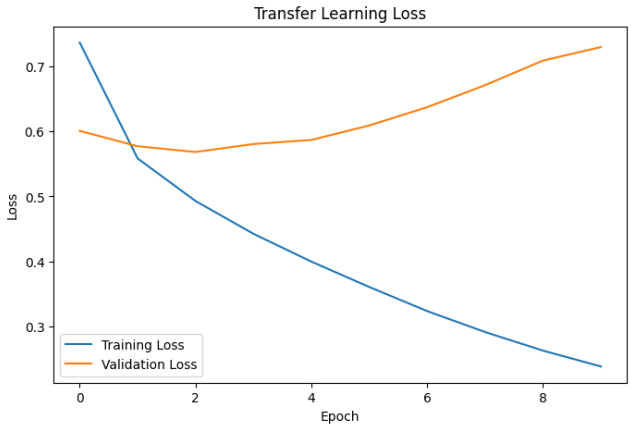
Both models were trained on the dataset, and their performance was monitored using training and validation metrics.

Training Accuracy Curve



The accuracy curve shows that the training accuracy gradually increased during training. The validation accuracy followed a similar trend, indicating that the model learned useful patterns from the dataset.

Training Loss Curve



The loss graph shows that the training loss decreased steadily during training, demonstrating that the model successfully minimized classification error.

Similar training curves were observed for the **transfer learning model**, which typically achieved higher accuracy faster due to pretrained feature extraction.

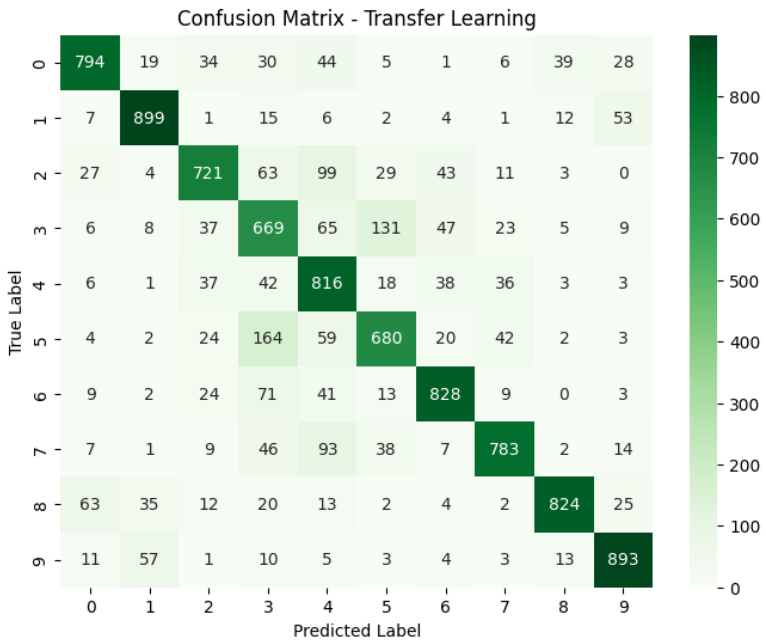
7. Model Evaluation

After training, both models were evaluated using the **test dataset**.

The following evaluation metrics were used:

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

Confusion Matrix



The confusion matrix shows how many images were correctly and incorrectly classified for each class.

Performance Metrics

The classification report provides precision, recall, and F1-score for each class.

These metrics help measure how well the model performs across all categories.

8. Model Comparison

The performance of the two models was compared using evaluation metrics.

Model	Description
Custom CNN	CNN architecture built from scratch
Transfer Learning	MobileNetV2 pretrained model

In most cases, the **transfer learning model performs better** because it uses features learned from a much larger dataset.

Transfer learning also reduces training time and improves generalization performance.

7. Model Evaluation

In this experiment, a **multiclass image classification system** was developed using Convolutional Neural Networks.

The CIFAR-10 dataset was used to train and evaluate the models. Data preprocessing techniques such as normalization, train-validation split, and data augmentation were applied to improve model performance.

A custom CNN architecture was first implemented to learn image features directly from the dataset. After that, a transfer learning approach using MobileNetV2 was applied to leverage pretrained features from the ImageNet dataset.

The experimental results showed that both models were able to classify images effectively. However, the transfer learning model generally achieved better performance due to the use of pretrained feature extraction.

Overall, this experiment demonstrates the effectiveness of CNNs for image classification tasks and highlights the advantages of transfer learning in deep learning applications.